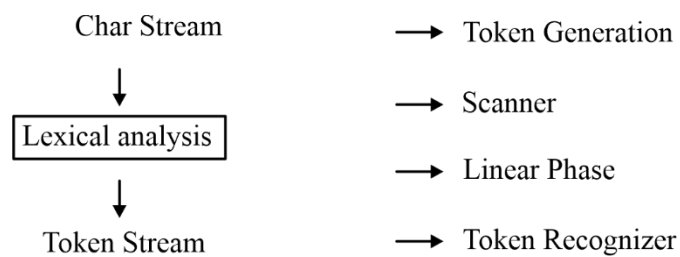


3

LEXICAL ANALYSIS

3.1 Lexical Analysis



3.1.1 Definition

Scan the whole program, char by char and produces the corresponding token.

- Also produces /Lexical Errors (if any).
- Functions of Lexical Analyzer.
 - (1) Scans all the character of the program.
 - (2) Token recognizer.
 - (3) Ignores the comment & spaces.
 - (4) Maximal Munch Rule [Longest Prefix Match].

Note:

The Lexical Analyser uses, the Regular Expression.
Prioritization of Rules.
Longest Prefix match .

- Lexeme → smallest unit of program or Logic.
- Token → internal representation of Lexeme.

3.1.2 Types of Token

- (1) Identifier
- (2) Keywords
- (3) Operators
- (4) Literals/Constants
- (5) Special Symbol

3.1.3 Token Separation

- (1) Spaces
- (2) Punctuation

3.1.4 Implementation

- LEX tool $\Rightarrow$ Lex.yy.e
- All identifiers will have entry in symbol Table/ LA, gives entries into the symbol Table.
Regular Expression $\rightarrow$ DFA $\rightarrow$ Lexical Analyzer

3.1.5 Find number of Tokens

- (1) `void main () {`
 `Printf("gate");`
 `}`
 {11 Tokens}
- (2) `int x, *P;`
 `x = 10; p = &x; x++;`
 [18 Tokens]
- (3) `int x;`
 `x = y;`
 `x == y;`
 [11 Tokens]
- (4) `int 1x23;` [Lexical Error]
- (5) `char ch = 'A';`
 [5 Token]
- (6) `char ch = 'A;`
 Lexical Error.
- (7) `char *p = "gate";`
 [6 Tokens]
- (8) `char * p = "gate;`
 Error.
- (9) `int x = 10;`
 `/* comment x = x + 1; ERROR`
 `x = x + 1; [11 Tokens]`

